



Profesora: Nancy Bernal Sánchez
Reforzamiento Ciclos-2024

Contenido Reforzamiento

★ Ciclos Repetitivos

- For

- While

★ Manejo de Errores

- Try - Except

Ciclos de Iteración/Repetitivos

Los ciclos de iteración o ciclos repetitivos son aquellos que nos permiten ejecutar un mismo código, sea ésta una o varias instrucciones de manera repetida, siempre y cuando se cumpla una condición. Esto nos permite que nuestro código sea más eficiente y atómico (compacto).

Estos ciclos de iteración, reciben varios nombres, entre ellos: cíclicos o bucles.

Python tiene definido dos estructuras de repetición y que son:

- **For**
- **While**





Bucle FOR

El ciclo For, nos permite recorrer los elementos de una secuencia sea esta una lista, tupla o cadena de caracteres, entre otras.

For, ejecuta una secuencia de instrucciones dependiendo de la cantidad de veces que se indique. Para ello, utilizaremos la función range que permite generar listas de números.

for
in
Python



Bucle FOR



Función range()

Esta función devuelve una **secuencia de números**, comenzando desde 0 de forma predeterminada y se incrementa en 1 y se detiene antes de un número especificado, es decir el último número no es inclusivo.



```
▶ print("Contemos.....")  
  for i in range(5):  
    print(i)
```



```
↳ Contemos.....  
0  
1  
2  
3  
4
```

Ejemplo:

Generar una secuencia de números del 0 al 100, luego debe mostrar cada elemento de la secuencia.

Bucle FOR



Función range()

También puede utilizar el range asignándolo a una variable:

```
▶ print("Pares hasta 10")  
r=range(11)  
for i in r:  
    if i%2==0:  
        print(i)
```

Pares hasta 10

0

2

4

6

8

10

Bucle FOR



Función range()

Puedo darle un parámetro de inicio y uno de fin:

Sintaxis:

`range(start, stop)`

Donde:

1. *Start*: Número donde comienza la secuencia.
2. *Stop*: Límite final de la secuencia, sin incluir el último número.



```
print("Inicio y Fin")
r=range(5,11)
for i in r:
    print(i)
```



```
Inicio y Fin
5
6
7
8
9
10
```

Bucle FOR

Función range()

Sintaxis:

range(start, stop, step)

Donde:

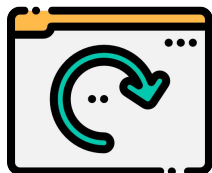
1. *Start*: Número donde comienza la secuencia.
2. *Stop*: Límite final de la secuencia, sin incluir el último número.
3. *Step*: Es el incrementando la secuencia.

Ejemplo:

Mostrar la secuencia de números entre 1 y 10 con incrementos de 2.

```
▶ print("Contando de 3 en 3")  
  r=range(9,24,3)  
  for i in r:  
      print(i)
```

```
Contando de 3 en 3  
9  
12  
15  
18  
21
```



Bucle FOR



Recorriendo Cadenas con for

Ejemplo

```
frase = "I Love PYTHON"  
for caracter in frase:  
    print(caracter)
```

I

L

o

v

e

P

Y

T

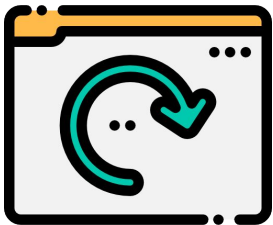
H

O

N

Para el ejemplo se utiliza un índice

llamado *caracter* dentro del ciclo FOR, con el propósito de recorrer y mostrar las letras que están contenidas en la *frase*.





Ejercitando con FOR

□ Ejercicio 1:

Ingresar por teclado 5 números enteros, luego debe indicar:

Cuántos números son mayores a cero

Cuántos números son menores a cero

Cuántos números son iguales a cero

□ Ejercicio 2:

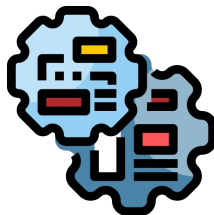
Ingrese por teclado 10 letras, indique cuántas de ellas son vocales y cuántas son consonantes.

□ Ejercicio 3:

Ingrese un número entero mayor a cero por teclado e indique si es o no "Primo".

□ Ejercicio 4:

Ingrese un número entero mayor a cero por teclado e indique si es o no "Perfecto".





Ejercitando con FOR

□ Ejercicio 5:

Genere las 10 primeras tablas de multiplicar, cada una de ellas de 1 a 10.

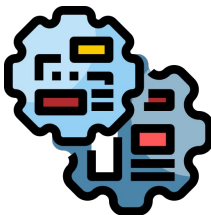
□ Ejercicio 6:

Genere las tablas de multiplicar de los primeros números pares entre 1 y 10.

□ Ejercicio 7:

Calcular la potencia de un número usando ciclos (no podrá usar el operador aritmético **), para ello el sistema solicitará por pantalla el valor de la base y el valor del exponente (ejemplo, base=4, exponente=2 entonces el resultado será 16), una vez realizado el cálculo se mostrará el resultado por una salida por pantalla mostrando la información de la siguiente manera ↓

“El resultado de la potencia de base 4 y exponente 2 es 16.”



Bucle WHILE



Ciclo que se repite **MIENTRAS** la condición es Verdadera.

1. La condición puede ser:

- Un literal
- El valor de una variable
- El resultado de una expresión
- El valor devuelto por una función

2. La o las instrucciones contenidas en el bloque **while**, contienen una sangría mayor, es decir, se aplica la indentación y este bloque finaliza cuando se encuentre la primera línea que se encuentra fuera de este ciclo.



Bucle WHILE



El ejemplo muestra lo siguiente:

1. La variable **a** se inicia con un valor **1**, por lo tanto, puede ingresar al ciclo while, dado que su condición es menor a 5 (verdadera).
2. Se imprime o muestra el valor de **a**, es decir **1**
3. Se incrementa en **1** el valor de **a**
4. Vuelve a la condición while y si la condición es verdadera, se ejecuta nuevamente el ciclo.
5. Finalmente, el resultado de este código es:



```
a = 1
while a < 5:
    print (a)
    a = a + 1
```

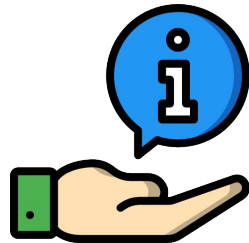
1
2
3
4

Ejercitando con WHILE



Ejercicio 1

Calcular el factorial de un número ingresado por teclado.



Factorial: Corresponde a multiplicaciones sucesivas entre el número 1 hasta el número ingresado y se representa con el signo de exclamación.

Ejemplos:

0!	:= 1	
1!	= 1	= 1
2!	= 1 · 2	= 2
3!	= 1 · 2 · 3	= 6
4!	= 1 · 2 · 3 · 4	= 24
5!	= 1 · 2 · 3 · 4 · 5	= 120
6!	= 1 · 2 · 3 · 4 · 5 · 6	= 720
7!	= 1 · 2 · 3 · 4 · 5 · 6 · 7	= 5,040
8!	= 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8	= 40,320
9!	= 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9	= 362,880

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

$$1! = 1$$

$$2! = 2 \times 1 = 2$$

$$3! = 3 \times 2 \times 1 = 6$$

$$4! = 4 \times 3 \times 2 \times 1 = 24$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Ejercitando con WHILE

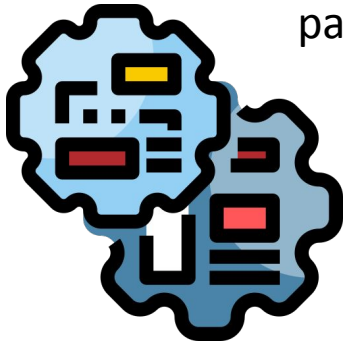


Ejercicio 2

Ingresa por teclado 5 números enteros positivos, súmelos y muestre el resultado de la suma.

Ejercicio 3

Solicite un valor numérico por pantalla, el valor debe ser impar, por lo que cada vez que se ingrese un número par, el sistema deberá mostrar un mensaje de error y volver a solicitar la variable, para este ejercicio use un ciclo de condición y el operador aritmético '%', al ingresar el número impar el sistema deberá mostrar por una salida de pantalla el valor multiplicado por 4.



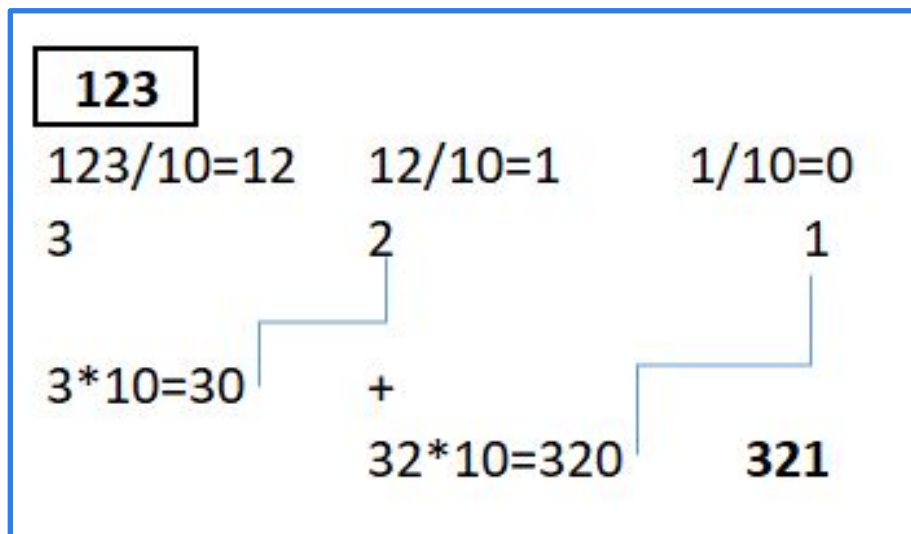
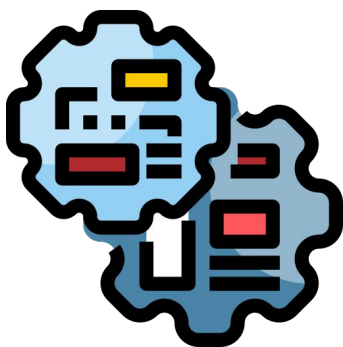
Ejercitando con WHILE



Ejercicio 4

Ingresa por teclado un número entero positivo entre 103 y 987, luego genere un proceso que permita invertir el número, aplicando las 4 operaciones matemáticas básicas (suma, resta, multiplicación y división). Finalmente, muestre el resultado.

Ejemplo:



Excepciones

- Las excepciones son una forma de controlar el comportamiento de un programa cuando éste genera un error.
- Cuando se produce el error, el programa se detiene.



Excepciones



Ejemplo 1: Realizar una división



```
a = 4  
b = 2  
c = a/b  
print (f"El resultado de la división es: {c}")
```

El resultado de la división es: 2.0



El resultado es 2, dado que **a** es dividendo, **b** el divisor y sus valores permiten desarrollar la división correctamente.

Excepciones

Ejemplo 2: Realizar una división por cero.

```
 a = 4
b = 0
print(a/b)
```

```
ZeroDivisionError                                Traceback (most recent call last)
<ipython-input-12-39f2597838d5> in <module>()
      1 a = 4
      2 b = 0
----> 3 print(a/b)

ZeroDivisionError: division by zero
```



Se produce un error de que no está permitido realizar una división por cero, por tanto se genera la excepción: **ZeroDivisionError: division by zero**

Excepciones

Ejemplo 3: Sumar números y texto.

```
▶ print(2 + "2")
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-13-96788054896b> in <module>()  
----> 1 print(2 + "2")  
  
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

SEARCH STACK OVERFLOW



Se produce un error y la excepción es `TypeError` y eso “ocurre cuando se aplica una operación con un dato de tipo inapropiado”

Excepciones

```
try:
    #acá se pone el código que podría desencadenar un
    #error en tiempo de ejecución
except:
    #y acá se pone la acción que se ejecutará
    #cuando se produzca el error
```

Existen excepciones específicas y que se “despiertan” frente a un error determinado.

Pero también puede implementarse una excepción genérica que engloba o contiene TODAS las anteriores.

Excepciones: Try - Except

```
▶ a = 5  
b = 0  
try:  
    c = a/b  
    print (f"El resultado es: {c}")  
except ZeroDivisionError:  
    print("No se ha podido realizar la división")
```

No se ha podido realizar la división

Dado el siguiente ejemplo, se presenta la división por cero, pero ahora tiene una estructura de código diferente.

Se consideran dos partes:

1. Try, la cual corresponde a la sección del código y esta podría lanzar la excepción.
2. Except, captura la excepción y hace que el programa no se caiga (Término técnico en programación).



Excepciones: Try - Except - Else



Al código compuesto con Try y Except, también podemos incluir el bloque **“else”**, el cual se ejecuta siempre y cuando no se cumpla ninguna excepción.

Ejemplo con except

```
▶ try:
    x = 2/0 # se fuerza a que ocurra la excepción
except:
    print("Ingresa al bloque except, Ocurrió una excepción")
else:
    print("Ingresa al bloque else. Sin excepción")
```

Ingresa al bloque except, Ocurrió una excepción

Ejemplo con else

```
▶ try:
    x = 2/2
except:
    print("Ingresa al bloque except. Ocurrió una excepción")
else:
    print("Ingresa al bloque else. Sin excepción")
```

Ingresa al bloque else. Sin excepción

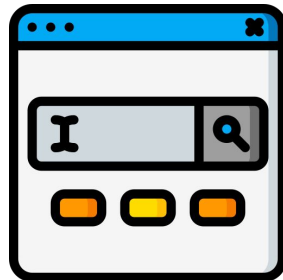


Apoyo con algunos link:
<https://uniwebsidad.com/libros/algoritmos-python/capitulo-12>

Excepciones: Investigar

Investiga como es:

- Uso de Raise
- Uso de Finally
- Excepción de OverflowError
- Excepción ValueError



Ejercitando con Todo

En un delivery de amasandería se venden 4 tipos de pan:

- Amasado \$1.500
 - Molde \$1.000
 - Baguette \$2.000
 - Integral \$3.000
-
- Determine el total a pagar por un cliente, el cual puede comprar diferentes tipos y cantidad de pan.
 - Si el total de la venta es superior a \$5000 el envío es gratis, si no se cobra el 10% adicional.
 - Muestre los mensajes correspondientes
 - Haga validaciones y maneje posibles errores

Toma el ejercicio de la Sandwichería el Duocano [visto en la EA1] y desarróllalo en Python.

**** Agrega un resumen de compra que incluya la cantidad de cada uno de los sandwich incluidos en una venta ****